IAM-11: Policy Persona Workshop

> **Series:** IAM — IAM Administration | **Notebook:** Bonus Workshop | **Created:** February 2026 | **Last Updated:** 10/13/2026

Overview

Every enterprise has distinct roles — application developers, SREs, platform engineers, security teams — each needing different levels of access to Dynatrace. A **persona-based permissions model** maps these roles to standardized policies across three domains: UI visibility, data access, and configuration control.

This workshop walks you through a structured, six-goal process to design that model for your organization. You will identify personas, align them with your identity provider, audit the Dynatrace UI for schema-level permissions, define data boundaries, and compose the policies that enforce your design.

> **Workshop Format:** This is a hands-on planning exercise that synthesizes concepts from **IAM-01** through **IAM-10**. Work through each goal sequentially, filling in the templates for your organization. Bring your AD team, security lead, and platform owners into the conversation.

Table of Contents

1. Tips and Key Principles
2. Procedure: Building Your Permissions Model
   - Goal 1: Identify Personas
   - Goal 2: Active Directory Alignment
   - Goal 3: Configuration Schema Audit
   - Goal 4: App Visibility by Persona
   - Goal 5: Data Requirements
   - Goal 6: Configuration Policy Design
3. Writing Policies
4. Naming Standards
5. Policy Flow and Examples
6. Validating Your Design
7. End-to-End Provisioning Script *(→ IAM-12)*
8. Appendix

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM enabled
Permissions	`account-user-management` to create/modify policies and group assignments
Prior Knowledge	**IAM-01** (Governance), **IAM-02** (SSO), **IAM-03** (Groups), **IAM-04** (Policies), **IAM-05** (Boundaries)
Administrative Access	Access to Active Directory or identity provider
Business Context	Knowledge of role definitions in your organization

> **Recommended Reading:** Complete **IAM-06** (User Lifecycle) and **IAM-10** (Templated Policy Assignments) before this workshop for the full context on provisioning and parameterized policies.

Learning Objectives

By the end of this workshop, you will:

- Identify and consolidate personas relevant to your Dynatrace deployment
- Align personas with Active Directory groups and a provisioning strategy
- Audit the Dynatrace UI to map configuration schemas to per-persona permissions
- Define data access requirements per persona using boundaries and segments
- Design configuration policies following the three-domain model (UI, Data, Config)
- Apply naming standards for policies, groups, and boundaries
- Validate your IAM design using DQL audit queries

Tips and Key Principles

Every company is different. The size of the user population and the division of responsibility can alter your approach. However, several rules are constant:

- **DEFAULT USER permissions apply to everyone** — Any permission granted to the default user group affects every user in your organization, including new hires provisioned via SCIM. Audit this group first. See **IAM-04** for policy evaluation order.
- **DENY trumps any ALLOW — use it sparingly** — A single DENY statement overrides all ALLOW statements for the same resource, regardless of which group it comes from. This means DENY in a broadly-scoped group (like Standard Users or Default Users) will **break composability** — a user who belongs to both Standard and Power User groups would be blocked by the Standard group's DENY, even though the Power User group grants ALLOW. **Use implicit deny instead:** simply omit the ALLOW for permissions you don't want to grant. Reserve explicit DENY only for narrowing a broad ALLOW within the *same* policy (e.g., "ALLOW write on all schemas EXCEPT this sensitive one"). This is covered in detail in **IAM-04: Policy Authoring**.
- **At minimum, distinguish "Standard" and "Power User" tiers** — Standard users view data and dashboards; power users can modify alerting thresholds, create workflows, and change configuration. Only trained individuals should hold power-user or admin access.
- **Grant the minimum necessary** — Every additional permission is an additional risk. Start restrictive and widen on request rather than starting wide and trying to lock down later.
- **Balance group count vs. administrative overhead** — More groups give finer control but require more maintenance (policy updates, membership reviews, SCIM mappings). Too few groups force you to over-grant. Aim for 5-10 groups in a typical mid-size deployment. See **IAM-03: Group Architecture** for patterns.
- **Use templated policies to scale** — When multiple teams need similar permissions with different scopes, use `${bindParam:...}` parameters instead of duplicating policies. See **IAM-10: Templated Policy Assignments**.

Procedure: Building Your Permissions Model

The six goals below guide you from raw role identification through to tested, deployable policies. Each goal produces a specific deliverable that feeds the next goal.

Goal 1: Identify Personas Within the Enterprise

A **persona** is a category of user defined by their job function and the Dynatrace capabilities they need. Start by listing every distinct role that interacts with Dynatrace, then consolidate overlapping roles into a manageable set of personas.

Step 1: Brainstorm Roles

Interview team leads, review existing AD groups, and check who currently has Dynatrace access. List every role, even if they seem similar:

- Application developer who monitors their own service
- Application developer who sets alerting thresholds for their team
- SRE responsible for multiple services in one business unit
- SRE responsible for cross-cutting infrastructure (networking, DNS, load balancers)
- Platform engineer managing OpenShift/Kubernetes clusters on-premise
- Cloud operations engineer managing AWS/Azure/GCP resources
- Security analyst reviewing access logs and vulnerability data
- Dynatrace administrator with full platform management

Step 2: Consolidate Into Personas

Group similar roles into a smaller set. Two developers who differ only by team can share a persona; an SRE who also manages infrastructure may need a separate one. Typical consolidation:

Raw Role	Consolidated Persona
App dev (view only)	**Standard User**
App dev (sets thresholds)	**Power User**
SRE for one org	**SRE - Scoped**
SRE cross-cutting	**SRE - Platform**
Security analyst	**Security Viewer**
DT admin	**Admin**

Step 3: Build Your Personas Table

Create a single-row table with one column per consolidated persona. This table will be your column header for the permission matrices in Goals 3-6. Fill each cell with a short label you will use consistently throughout this workshop and in your policy/group names.

Standard User	Power User	SRE - Scoped	SRE - Platform	Security Viewer	Admin
View dashboards, run queries	*Modify alerting, create notebooks*	*Full ops for assigned apps*	*Infrastructure-wide ops*	*Read-only audit and security data*	*Full platform management*

> **Tip:** Keep it to 4-7 personas. Fewer than 4 usually means you are over-granting; more than 7 becomes hard to maintain. You can always split a persona later.

Deliverable

A finalized personas table with clear labels and one-line scope descriptions.

Finding Available Configuration Schemas

Before auditing permissions, you need to know which schemas exist. Every settings screen in the **classic (Gen2) Dynatrace UI** is backed by a schema (e.g., `builtin:anomaly-detection.metric-events`). These schemas are still in use today but will eventually be phased out as Gen3 **apps** fully replace them. Gen3 apps have their own independent access control via `app:` permissions (covered in Goal 4).

Until that transition is complete, you still need to audit and manage classic schema-level permissions. You can discover schemas in three ways:

1. **In the UI** — Navigate to **Settings** and open any settings page. The **Schema ID** and **Schema Group** appear in the upper-right corner of the page.

2. **Via search** — Use the settings search bar to find schemas by keyword (e.g., search "alerting" to find all alerting-related schemas).
3. **Via the API** — Call `GET /api/v2/settings/schemas` (requires `settings.read` scope) to get the full list programmatically.

The following code retrieves all schemas programmatically. Paste it into a **notebook code cell** (JavaScript/TypeScript) to run it directly:

```
import { settingsSchemasClient } from "@dynatrace-sdk/client-classic-environment-v2";
export default async function() {
    const includeCriteria = ['']; // filter by raw text here
    const data = await settingsSchemasClient.getAvailableSchemaDefinitions();
    const schemaSet = data.items;
    const schemas = schemaSet.filter(schema =&gt;
        includeCriteria.some(substring =&gt; schema.schemaId.includes(substring))
    );
    const result = [];
    for (const schema of schemas) {
        const id = schema.schemaId;
        const displayName = schema.displayName;
        result.push({ id, displayName });
    }
    return result;
}
```

> **Tip:** Set `includeCriteria` to filter results — e.g., `['anomaly', 'alerting']` to find only anomaly detection and alerting schemas. Use `['']` to return all schemas.

Goal 2: Align with Active Directory Strategy

Each consolidated persona needs a corresponding group in your identity provider. Before creating groups, answer these questions with your AD/IdP team:

Group logistics:
* Are there limits on the number of groups you can create?
* Must groups live under a specific enterprise application registration?
* Will each application team own their own AD groups, or will a central team manage them?
* Who approves group membership requests?

SAML/OIDC integration:
* What attributes does your IdP return in the SAML assertion or OIDC token?
* Can you include group membership as a claim? (Required for automated group mapping — see **IAM-02: SSO Authentication**)

Provisioning method (see **IAM-06: User Lifecycle**):
* **SCIM** — Automated sync from your IdP. Best for large deployments.
* **JIT (Just-In-Time)** — Users are created on first SSO login. Simpler but less control.
* **Manual** — Invite by email via the Dynatrace UI. Only suitable for admin/break-glass accounts.

Deliverable

A mapping table showing each persona, its AD group, and the provisioning method:

Persona	AD Group Name	Provisioning	Approval	Notes
Standard User	`DT-Standard-Users`	SCIM	Auto	Default for all Dynatrace users
Power User	`DT-Power-Users`	SCIM	Manager	Requires Dynatrace training completion
SRE - Scoped	`DT-SRE-<OrgName>`	SCIM	Team lead	One group per org/team
SRE - Platform	`DT-SRE-Platform`	SCIM	Platform lead	Cross-cutting infrastructure
Security Viewer	`DT-Security-Viewers`	SCIM	Security lead	Read-only audit access
Admin	`DT-Admins`	Manual	CISO + quarterly review	Break-glass; see **IAM-08**

Goal 3: Audit the Dynatrace UI for Configuration Permissions

This goal covers **classic (Gen2) settings schemas** — the configuration screens found under Settings in Dynatrace. These schemas are still actively used but are being progressively replaced by Gen3 apps with independent access control (see Goal 4 for app-level permissions).

Walk through the Dynatrace settings UI and catalog which configuration schemas each persona needs access to. This is the most time-intensive goal — budget 1-2 hours for a thorough audit.

How to build the audit table:

1. Open Dynatrace **Settings** and navigate through each category: **Host** → **Process** → **Service** → **Web Applications** → **Synthetic** → **Anomaly Detection** → etc.
2. For each settings screen, note the **Schema ID** (top-right corner) and the **Schema Group** (breadcrumb path).
3. Add a row to the table below and assign a permission level per persona:
   - **none** — Persona cannot see this setting at all
   - **read** — Persona can view but not modify
   - **write** — Persona can read and modify

> **Tip:** Start with the defaults: Standard Users get **read** on most schemas and **none** on sensitive ones. Admins get **write** on everything. Then adjust the middle tiers.

Reference: Schema Audit Table

Create your own table and update the personas for your organization. Values that begin `group:` are schema **groups** — scope them with `settings:schemaGroup = "group:…"`, not `settings:schemaId`. Below is a starter template with common **classic (Gen2) schemas**. As Gen3 apps replace these settings screens, update your policies to use `app:` permissions instead:

Functionality	Schema ID / Group	Schema Group	Standard User	Power User	SRE - Scoped	SRE - Platform	A
Host monitoring	`group:host-monitoring`	`group:host-monitoring`	read	read	write	write	
Containers	`group:processes-and-containers.containers`	`group:processes-and-containers`	read	read	write	write	
Preferences	`group:preferences`	`group:preferences`	read	write	write	write	
General monitoring	`group:monitoring`	`group:monitoring`	read	write	write	write	
Process group state	`builtin:process-group.monitoring.state`	N/A	read	write	write	write	
OneAgent features	`builtin:oneagent.features`	`group:preferences`	read	write	write	write	
Anomaly detection	`group:anomaly-detection`	`group:anomaly-detection.infrastructure`	read	write	write	write	
OneAgent updates	`builtin:deployment.oneagent.updates`	`group:updates`	read	write	write	write	
OS services monitoring	`builtin:os-services-monitoring`	`group:monitoring`	read	read	read	write	
Log monitoring	`group:log-monitoring`	`group:log-monitoring.ingest-and-processing`	read	read	read	write	
Network & discovery	`group:network-and-discovery`	`group:monitoring`	read	read	read	write	
Failure detection	`group:failure-detection`	`group:failure-detection`	read	write	write	write	

Functionality	Schema ID / Group	Schema Group	Standard User	Power User	SRE - Scoped	SRE - Platform	A
Privacy settings	`group:privacy-settings`	`group:preferences`	read	read	write	write	
RUM settings	`group:rum-settings`	`group:rum-settings`	read	write	write	write	
RUM injection	`group:rum-injection`	`group:rum-injection`	read	read	read	read	
Web & mobile monitoring	`group:web-and-mobile-monitoring`	`group:web-and-mobile-monitoring`	read	write	write	write	

Gen3 Platform Permissions

Beyond classic settings schemas, Gen3 introduces platform-level permissions that control access to newer capabilities:

Capability	Standard User	Power User	SRE	Admin
Anomaly Detector — Create	none	none	write	write
Extensions — Create	none	none	read	write
Site Reliability Guardian — Read	read	read	read	read
Synthetic HTTP & NAM — Create	none	write	write	write
Segments — Read	read	read	read	read
Segments — Create/Edit	none	none	write	write

Deliverable

A completed schema audit table with a permission level in every cell. This drives your **configuration policies** in Goal 6.

Goal 4: Audit the UI for Each Persona's App Visibility

Separate from configuration schemas, determine which **Dynatrace apps and views** each persona should see in the navigation. Hiding apps that a persona does not need reduces cognitive load and prevents accidental changes.

Walk through the Dynatrace left-hand navigation and fill in this matrix:

App / View	Standard User	Power User	SRE	Admin
Dashboards	View shared	View + Create	View + Create	Full
Notebooks	View shared	View + Create	View + Create	Full
Workflows	Hidden	View	View + Create	Full
Extensions	Hidden	Hidden	View	Full
Settings	Hidden	Limited	Scoped	Full
Account Management	Hidden	Hidden	Hidden	Full
Security Overview	Hidden	Hidden	View	Full
Release Management	View	View	View + Manage	Full

Gen3 UI policies use three services together:

- `app-engine:apps:run` — Controls whether a persona can **see** an app in the navigation. Works for both Gen3 apps and Gen2 classic screens. Use `shared:app-id` to target specific apps.
- `document:` — Controls what the persona can **do** with dashboards and notebooks (read, write, create, delete, share)
- `environment:roles:viewer` — Grants baseline read access to the environment. For a persona whose classic entity view is limited by a management-zone boundary, grant it **only** in that boundary-scoped policy (see *Writing Policies*) — an unbounded grant here would undo the boundary

```
// Grant baseline environment viewer role — unscoped personas only;
// MZ-scoped personas get it from their boundary-scoped Data-Classic policy instead
ALLOW environment:roles:viewer;

// Make specific Gen3 apps visible — apps not listed here are implicitly hidden
ALLOW app-engine:apps:run WHERE shared:app-id IN ("dynatrace.notebooks", "dynatrace.dashboards");

// Grant access to a Gen2 (classic) screen
ALLOW app-engine:apps:run WHERE shared:app-id IN ("dynatrace.classic.synthetic");

// Allow viewing documents only — write/create/delete are implicitly denied
// (do NOT use DENY here — it would override ALLOWs from other groups)
ALLOW document:documents:read;
```

> **Key distinction:** `app-engine:apps:run` controls app visibility for both Gen3 apps (e.g., `dynatrace.dashboards`) and Gen2 classic screens (e.g., `dynatrace.classic.synthetic`). `document:documents:read` lets the persona open and view dashboard/notebook content. Both are needed for a persona to use dashboards.
>
> **The classic-screen grant is a transitional pattern.** `dynatrace.classic.*` apps are being retired — `dynatrace.classic.synthetic` is replaced by the Synthetic app — so a persona matrix built on classic app-ids has a shelf life. Grant them where a persona genuinely still works in a classic screen, and record each one as migration work rather than as a permanent entitlement.
>
> The same mechanism runs in reverse, and is the more durable move: a boundary of `shared:app-id not startsWith "dynatrace.classic"` restricts a group to the latest apps entirely. Dynatrace documents that as the way to prevent classic access after upgrading — worth adding to your matrix as the end state each persona is heading toward.

Discovering Installed Apps

To build your visibility matrix, you need to know which apps are installed. Use the **App Engine Registry API** to get a complete list of active, runnable apps in your environment:

```
GET /platform/app-engine/registry/v1/apps?include-deactivated=false&amp;include-non-runnable=false&amp;include-all-app-versions=false
```

This requires a bearer token with appropriate scopes. The response includes every app's ID (e.g., `dynatrace.dashboards`, `dynatrace.notebooks`, `dynatrace.classic.synthetic`) — use these IDs directly in your `app-engine:apps:run WHERE shared:app-id` policy statements.

See **IAM-04: Policy Authoring** for the full `app-engine:apps:run` and `document:` permission syntax.

Deliverable

A completed app visibility matrix. This becomes the basis for your **UI policies**.

Goal 5: Determine Data Requirements Per Persona

Beyond configuration and UI access, determine what **data** each persona should see. This drives your **data policies** and **boundary** design (see **IAM-05: Boundary Design Patterns**).

Consider five dimensions:

Dimension	Question to Answer
Scope	Should this persona see data from all applications, or only their team's?
Data domains	Which data types: metrics, logs, traces, events, business events?
Boundaries	Which Dynatrace boundaries restrict their data view?
Sensitive fields	Do they need access to fieldsets like `builtin-sensitive-spans`? (See Appendix)
Bucket scoping	Should data access be limited to specific Grail buckets (e.g., `prod_logs` only)?

Data Access Matrix Template

Persona	Metrics	Logs	Traces	Events	Gen3 Boundary (Grail data)	Gen2 Boundary (Classic entities)	Sensitive Fields
Standard User	All	Own apps	Own apps	All	`storage:dt.security_context IN ("team")`	`environment:management-zone IN ("Team-MZ")`	No
Power User	All	All	All	All	None (env-wide)	None (env-wide)	No
SRE - Scoped	All	Own org	Own org	All	`storage:dt.security_context IN ("org")`	`environment:management-zone IN ("Org-MZ")`	Yes
SRE - Platform	All	All	All	All	None (env-wide)	None (env-wide)	Yes
Security Viewer	None	None	None	Audit + security events	None — the data policy scopes itself (below)	None	No
Admin	All	All	All	All	None (env-wide)	None (env-wide)	Yes

> **Security Viewer data policy.** Platform audit records are `AUDIT_EVENT` records in `dt.system.events`, stored in the `dt_system_events` bucket — there is no separate audit-log bucket. Grant: `> ALLOW storage:buckets:read WHERE storage:bucket-name = "dt_system_events"; > ALLOW storage:system:read WHERE storage:event.kind = "AUDIT_EVENT"; > ALLOW storage:buckets:read WHERE storage:table-name = "security.events"; > ALLOW storage:security.events:read; >`
>
> **Scope the audit read inside the policy, not with a boundary.** A `storage:event.kind` boundary does not do this job: `storage:buckets:read` accepts only `storage:bucket-name`, `storage:table-name` and `storage:query-consumption`, so the boundary leaves it unconditional, and on `storage:security.events:read` it would narrow security events to kind `AUDIT_EVENT` rather than leave them readable. The `WHERE` clauses above already limit the persona to audit records plus security events.
>
> **Sources:** Audit logs on Grail (DT docs) — *"ALLOW storage:system:read WHERE storage:event.kind = "AUDIT_EVENT""*, IAM policy statements (DT docs) — `storage:buckets:read`: *"Required additionally to a table permission."*, Policy boundaries (DT docs) — *"Only conditions added to service configuration will be applied to relative permissions."*

Deliverable

A completed data access matrix. This becomes the basis for your **data policies** and **boundary assignments**.

Goal 6: Design Configuration Policies Per Persona

With your schema audit (Goal 3), app visibility (Goal 4), and data requirements (Goal 5) complete, translate each row of those matrices into policy statements.

For each persona, create one policy per domain (UI, Data, Config). Use the schema IDs from your Goal 3 audit table directly in the `WHERE` clauses.

Configuration Policy Examples

```
// GLOBAL-Standard-Config
// Standard Users: read-only access to all settings
// Write access is implicitly denied — no ALLOW means no access
ALLOW settings:objects:read, settings:schemas:read;
```

```
// GLOBAL-PowerUser-Config
// Power Users: read all settings, write only alerting and notifications
ALLOW settings:objects:read, settings:schemas:read;
ALLOW settings:objects:read, settings:objects:write, settings:schemas:read
  WHERE settings:schemaId startsWith "builtin:alerting.profile";
ALLOW settings:objects:read, settings:objects:write, settings:schemas:read
  WHERE settings:schemaId startsWith "builtin:problem.notifications";
ALLOW settings:objects:read, settings:objects:write, settings:schemas:read
  WHERE settings:schemaGroup = "group:preferences";
```

```
// ORG1-SRE-Config (use templated policy from IAM-10 if multiple orgs)
// SREs: full settings access scoped by schema group
ALLOW settings:objects:read, settings:schemas:read;
ALLOW settings:objects:read, settings:objects:write, settings:schemas:read
  WHERE settings:schemaGroup = "group:host-monitoring";
ALLOW settings:objects:read, settings:objects:write, settings:schemas:read
  WHERE settings:schemaGroup = "group:anomaly-detection";
ALLOW settings:objects:read, settings:objects:write, settings:schemas:read
  WHERE settings:schemaGroup = "group:failure-detection";
ALLOW settings:objects:read, settings:objects:write, settings:schemas:read
  WHERE settings:schemaGroup = "group:processes-and-containers";
```

> **Schema IDs and schema groups are different conditions.** Schema IDs begin `builtin:` or `app:` — match them with `settings:schemaId`. A `group:` value is a **schema group**, matched with `settings:schemaGroup` (operators `=` and `IN` only). A `settings:schemaId startsWith "group:…"` statement is accepted but matches no schema, so it silently grants nothing.
>
> **Sources:** IAM policy reference (DT docs) — *"settings:schemaGroup - A schema group that allows to address multiple individual schemas at once."*
>
> **Scaling tip:** If you have multiple orgs that need the same SRE config policy with different data scopes, create a **templated policy** with `${bindParam:team}` and bind it per-group. See **IAM-10**.

Deliverable

A policy statement (or template reference from **IAM-10**) for each persona covering configuration access.

> **Binding count:** Scoped personas (Standard User, SRE-Scoped) need **4 policy bindings**: UI + Gen3-Data + Gen2-Data (Classic entity) + Config. Unscoped personas (Power User, SRE-Platform, Admin) and Security Viewer — whose data policy scopes itself — need **3 bindings**: UI + Data + Config. Budget 18-24 policy bindings for a typical 6-persona deployment. See **Writing Policies** for the complete Standard User example showing all four bindings.

Writing Policies

Now that you have deliverables from all six goals, assemble the actual policies. Keep each policy focused on a single domain. Resist the temptation to combine UI, data, and config permissions into one large policy — separate policies are easier to audit, reuse, and update independently.

The Three Policy Domains

Every persona needs at least one policy in each domain:

UI Policy — Controls which Dynatrace apps the persona can see in the navigation (`app-engine:apps:run` service with `shared:app-id`), baseline environment access (`environment:roles:viewer` — for unscoped personas only; scoped personas get it from their boundary-scoped Data-Classic policy), and what they can do with documents like dashboards and notebooks (`document:` service).

Data Policy — Controls which data (metrics, logs, traces, events) is returned when the persona queries the platform. Scoped by boundaries, buckets, or security context.

```
//Grail Data Access
ALLOW storage:buckets:read,
storage:logs:read,
storage:metrics:read,
storage:bizevents:read,
storage:events:read,
storage:spans:read,
storage:user.events:read,
storage:entities:read;

//Classic Data Access
ALLOW environment:roles:viewer;
```

Config Policy — Controls which settings and configurations the persona can read or modify. Scoped by schema ID or schema group.

```
//Allow Extension Config
ALLOW extensions:definitions:read, extensions:definitions:write, extensions:configurations:read,
extensions:configurations:write, extensions:configuration.actions:write;

//App settings for SRE Guardian
ALLOW app-settings:objects:read, app-settings:objects:write WHERE shared:app-id IN ("dynatrace.site.
reliability.guardian");

//Builtin Write Access
//Allowing write for specific Schema IDs and read for all
ALLOW settings:objects:read, settings:schemas:read;
ALLOW settings:objects:read, settings:objects:write WHERE settings:schemaId IN ("builtin:crashdump.
analytics", "builtin:host.process-groups.monitoring-state","builtin:process-group.monitoring.state",
"builtin:deployment.oneagent.updates","builtin:url-based-sampling","builtin:alerting.
connectivity-alerts", "builtin:oneagent.features", "builtin:process-group.monitoring.state",
"builtin:availability.process-group-alerting","builtin:alerting.connectivity-alerts","builtin:settings.
mutedrequests", "builtin:settings.subscriptions.service","builtin:rum.mobile.request-errors");

//Allowing Full Workflow Functionality
ALLOW automation:workflows:admin, automation:workflows:read, automation:workflows:write,
automation:workflows:run, automation:rules:read, automation:rules:write, automation:calendars:read,
automation:calendars:write;

//Allowing Install of Apps
ALLOW app-engine:functions:run, app-engine:apps:run, app-engine:apps:install;

//Allowing Certain capabilities for buckets
ALLOW storage:bucket-definitions:read,  storage:bucket-definitions:truncate;

//Allowing segment definition and share
ALLOW storage:filter-segments:read, storage:filter-segments:write, storage:filter-segments:share,
storage:filter-segments:delete;

//Allow full openpipeline access
ALLOW openpipeline:configurations:read, openpipeline:configurations:write;

```

Assembling a Complete Persona Policy Set

For each persona, compose three policy documents using your deliverables from Goals 3-6:

Step	Input	Output Policy
1	Goal 4 app visibility matrix	`<SCOPE>-<PERSONA>-UI`
2a	Goal 5 Gen3 Grail data requirements	`<SCOPE>-<PERSONA>-Data` (Gen3)
2b	Goal 5 Gen2 Classic entity requirements	`<SCOPE>-<PERSONA>-Data-Classic` (Gen2; omit if env-wide)
3	Goal 3 schema audit + Goal 6 config design	`<SCOPE>-<PERSONA>-Config`

Example: Complete Standard User policy set

```
// GLOBAL-Standard-UI
// No environment:roles:viewer here — Standard Users are MZ-scoped, so the role is granted
// only in GLOBAL-Standard-Data-Classic, under the MZ boundary. An unbounded grant in this
// policy would give the same role environment-wide and undo that boundary.
// App visibility — only these apps appear in the navigation
// All other apps (Settings, Account Management, etc.) are implicitly hidden
ALLOW app-engine:apps:run WHERE shared:app-id IN ("dynatrace.dashboards", "dynatrace.notebooks");
// Document permissions — read only; write/create/delete are implicitly denied
ALLOW document:documents:read;
```

```
// GLOBAL-Standard-Data (Gen3 Grail data — pair with a Gen3 boundary)
// Gen3 boundary: storage:dt.security_context IN ("team"); settings:dt.security_context IN ("team");
// storage:buckets:read is required in addition to every table permission
ALLOW storage:buckets:read;
ALLOW storage:logs:read WHERE storage:dt.security_context IN ("${bindParam:team}");
ALLOW storage:spans:read WHERE storage:dt.security_context IN ("${bindParam:team}");
ALLOW storage:metrics:read;
ALLOW storage:events:read;
```

```
// GLOBAL-Standard-Data-Classic (Gen2 Classic entity access — pair with a Gen2 boundary)
// Gen2 boundary: environment:management-zone IN ("Team-MZ");
// Attach this binding SEPARATELY from the Gen3 binding above, with its own boundary. Never attach
// two boundaries to one binding: each is evaluated separately, so each leaves the other's domain unscoped.
// Remove this binding when Management Zone retirement is complete.
ALLOW environment:roles:viewer;
```

```
// GLOBAL-Standard-Config
// Read-only access to all settings — write is implicitly denied
ALLOW settings:objects:read, settings:schemas:read;
```

> **Why the viewer role lives only in Data-Classic.** A boundary narrows only the binding it is attached to. If the UI policy also granted `environment:roles:viewer` unbounded, the user would hold the role environment-wide through that binding and the MZ boundary on Data-Classic would restrict nothing.
>
> **Sources:** Policy boundaries (DT docs) — *"When more than one boundary applies to a policy, then effective statements are calculated for each boundary separately"*. **Derived:** the unbounded-grant conclusion combines per-binding boundaries with additive ALLOWs.
>
> **Why no DENY?** These policies rely on **implicit deny** — if a permission is not ALLOWed, it is denied by default. This is critical for composability: a Standard User who is *also* added to a Power User or SRE group will correctly inherit the additional ALLOWs from those groups. If this policy contained `DENY settings:objects:write`, that DENY would **override** the Power User's `ALLOW settings:objects:write` and the user would be locked out of configuration — even though the intent was to grant them elevated access.
>
> **When to use DENY:** Reserve DENY for narrowing a broad ALLOW within the *same* policy — for example, "ALLOW write on all schemas EXCEPT this sensitive one." Never place DENY in a broadly-assigned group (like Default Users) where it would block permissions granted by other groups.
>
> **Review checkpoint:** Before binding policies to groups, have a second person review each policy statement. A misplaced DENY in the default user group can lock everyone out of a capability.

Naming Standards

Consistent naming is critical for maintainability. When you have 20+ policies across 6 personas, clear names prevent confusion during audits, incident response, and onboarding new admins.

Policy Naming Convention

```
&lt;SCOPE&gt;-&lt;PERSONA&gt;-&lt;DOMAIN&gt;
```

Component	Description	Examples
Scope	Organizational scope of the policy	`GLOBAL`, `LOB1`, `TEAM-PLATFORM`, `ORG2`
Persona	The consolidated persona name	`Standard`, `PowerUser`, `SRE`, `Admin`
Domain	The policy domain	`UI`, `Data`, `Config`

Policy Name	What It Controls
`GLOBAL-PowerUser-UI`	App visibility for power users (all orgs)
`LOB1-Standard-Data`	Data access scoped to LOB1 applications
`ORG2-SRE-Config`	Configuration write access for SREs in Org 2
`GLOBAL-Admin-Config`	Full configuration access for admins

Group Naming Convention

Groups follow a parallel pattern (see **IAM-03: Group Architecture**):

```
DT-&lt;SCOPE&gt;-&lt;PERSONA&gt;
```

Group Name	Assigned Policies
`DT-GLOBAL-PowerUsers`	`GLOBAL-PowerUser-UI`, `GLOBAL-PowerUser-Data`, `GLOBAL-PowerUser-Config`
`DT-LOB1-StandardUsers`	`LOB1-Standard-UI`, `LOB1-Standard-Data`, `LOB1-Standard-Config`
`DT-ORG2-SREs`	`ORG2-SRE-UI`, `ORG2-SRE-Data`, `ORG2-SRE-Config`

Boundary Naming Convention

This workshop keeps Gen3 (Grail data) and Gen2 (Classic entities) conditions in **separate named boundaries**, one per binding, and uses a suffix to distinguish them. That is a design choice, not a platform rule: a single boundary that mixes `environment:management-zone` and `storage:` conditions is documented and works. The rule that matters is the opposite one — **never attach two boundaries to one binding**, because effective statements are calculated for each boundary separately and each leaves the other's domain unscoped:

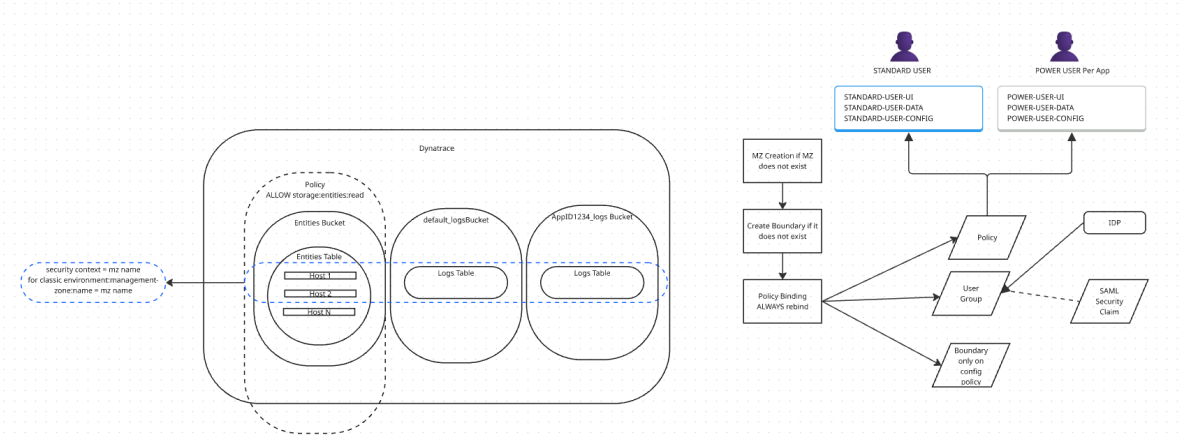
```
bnd-gen3-&lt;ISOLATION-TYPE&gt;-&lt;SCOPE&gt;   ← storage:/settings: domain conditions
bnd-gen2-&lt;ISOLATION-TYPE&gt;-&lt;SCOPE&gt;   ← environment: domain conditions
```

Boundary Name	Domain	Conditions
`bnd-gen3-team-checkout`	Gen3 (Grail)	`storage:dt.security_context IN ("checkout"); settings:dt.security_context IN ("checkout");`
`bnd-gen2-team-checkout`	Gen2 (Classic)	`environment:management-zone IN ("Checkout-Team");`
`bnd-gen3-env-production`	Gen3 (Grail)	`storage:dt.security_context IN ("production");`
`bnd-gen2-env-production`	Gen2 (Classic)	`environment:management-zone IN ("Production-MZ");`

> **Two bindings per scoped group:** attach `bnd-gen3-team-*` to the Gen3 data policy binding, and `bnd-gen2-team-*` to the Gen2 Classic entity policy binding. See **IAM-05: Boundary Design Patterns** for the full two-binding pattern. (Security Viewer needs no boundary — its data policy scopes the audit read itself; see Goal 5.)
>
> **Sources:** Policy boundaries (DT docs) — *"When more than one boundary applies to a policy, then effective statements are calculated for each boundary separately"*; the page's own `Kubernetes` example boundary combines `environment:management-zone startsWith "[Kubernetes]";` with `storage:k8s.namespace.name IN ("DEV","PREPROD");`.

Policy Flow and Examples

The following diagram shows the end-to-end flow from persona identification through policy assignment, including automation with Monaco:



> **Next Step:** Once your policies are defined and bound, use **IAM-10: Templated Policy Assignments** to parameterize policies that repeat across teams or scopes, and manage everything as code with Monaco.

Validating Your Design

After binding your policies to groups, validate that the implementation matches your design. See **IAM-07: Audit Logging and Compliance** for the full audit query catalog.

Key Validation Checks

Check	How	Reference
Policies bound to correct groups	Review the account audit log (Account Management > Settings > Audit log) for binding changes	**IAM-07** Section 1
No unintended DENY overrides	Review effective permissions per group; test as each persona	**IAM-04** evaluation order
Users provisioned into correct groups	Review group membership changes in the account audit log	**IAM-07** Section 1
Data boundaries working	Run the same DQL query as different personas and compare results	**IAM-05** boundary testing
SCIM sync functioning	Monitor SCIM provisioning events for errors	**IAM-06** lifecycle events

Group, policy and binding changes are account-level and are recorded in the account audit log, not in environment audit events. The DQL query below shows IAM API calls made through this environment — useful for spotting provisioning automation, not a change log:

```
// IAM API activity in this environment (last 7 days)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in `logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type (GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"), authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path), origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE", "UPDATE"}) and contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| fields timestamp, user.id, event.type, resource
| sort timestamp desc
| limit 25
```

End-to-End Provisioning Script

The complete provisioning script — including OAuth setup, group creation, policy creation across all three domains, and templated data bindings — lives in **IAM-12: API Provisioning & Validation Scripts**.

IAM-12 includes:

Script	Purpose
Script 1: End-to-End Provisioning	Creates groups, policies (UI + Config + Data), and bindings for all personas
Script 2: Policy & Binding Report	Queries the API and displays all policies with bindings and template parameter values
Script 3: Cleanup	Removes test resources by name prefix
DQL Validation Queries	Audit policy changes, verify security contexts, check bucket assignments
API Reference	Endpoint summary, valid permissions, common quirks
Troubleshooting	Error codes, silent-failure checklist, templated-policy debugging

> **Why a separate notebook?** The provisioning scripts are operational tooling — they change every time you add a persona or team. Keeping them in IAM-12 lets you iterate on the scripts without touching the workshop design in this notebook.

Appendix

Fieldsets

Tables can have sensitive fields visible only to users with the right permissions. A **fieldset** is a named collection of sensitive fields defined at the bucket, table, or tenant scope.

Dynatrace ships three predefined fieldsets — two for spans and one for `user.events` / `user.sessions` — and you can define custom fieldsets scoped to buckets or tables. Field permissions work with Smartscape, not with classic entities.

Fieldset	Contents
`builtin-request-attributes-spans`	Dynamically generated from Request Attributes
`builtin-sensitive-spans`	`client.ip`, `db.connection_string`, `http.request.header.referer`, `url.full`, `url.query`, `db.bind.parameter`

Fieldset	Contents
`builtin-sensitive-user-events-and-sessions`	Sensitive fields in `user.events` and `user.sessions`: `client.ip`, `dt.rum.user_tag`, `geo.location.latitude`, `geo.location.longitude`

The RUM fieldset is in no default policy — grant it explicitly: `ALLOW storage:fieldsets:read WHERE storage:fieldset-name = "builtin-sensitive-user-events-and-sessions";`

> **Sources:** Permissions in Grail (DT docs) — *"The three predefined fieldsets are:"*; *"You can use the field permissions with smartscape, but not with entities."*; RUM data access controls best practices (DT docs) — *"This permission is not included in any policy by default and must be assigned explicitly"*.

Fieldset Permissions

To read sensitive fieldsets, a user needs an explicit `storage:fieldsets:read` permission. Without it, the query still runs (no 403), but *"sensitive fields won't be shown in the result."*

Unconditional (access to all fieldsets):

```
ALLOW storage:fieldsets:read;
```

Conditional (scoped to specific buckets, tables, or fieldsets):

```
// By bucket
ALLOW storage:fieldsets:read
  WHERE storage:bucket-name IN ("default_spans", "sensitive_spans");

// By table
ALLOW storage:fieldsets:read
  WHERE storage:table-name = "spans";

// By fieldset name
ALLOW storage:fieldsets:read
  WHERE storage:fieldset-name = "builtin-sensitive-spans";
```

> **Note:** In community practice, an unconditional fieldset permission is treated as overriding conditional ones, by analogy with table permissions — for those the page states *"If there is an unconditional table permission in any other policy available for a user, the `WHERE` clause is irrelevant"*. Verify for fieldsets in your tenant.
>
> **Sources:** Permissions in Grail (DT docs) — *"If you don't have sufficient permissions, sensitive fields won't be shown in the result."*

References

IAM Series Cross-References:

Notebook	Relevance to This Workshop
IAM-01: IAM Governance Foundations	Governance models and organizational patterns
IAM-02: SSO Authentication	SAML/OIDC setup for AD integration (Goal 2)
IAM-03: Group Architecture	Group hierarchy patterns for persona mapping
IAM-04: Policy Authoring	Policy syntax, `app:` permissions, evaluation order
IAM-05: Boundary Design Patterns	Data segmentation for per-persona scoping
IAM-06: User Lifecycle	SCIM provisioning, JIT, offboarding
IAM-07: Audit Logging and Compliance	DQL audit queries for validation
IAM-08: Multi-Environment IAM	Break-glass accounts, cross-env patterns
IAM-09: Troubleshooting	Permission denied debugging

Notebook	Relevance to This Workshop
IAM-10: Templated Policy Assignments	Parameterized policies for scaling

Dynatrace Documentation:

- Identity and Access Management
- Policy Overview
- Account Management API

⚠️ **DISCLAIMER**: This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.